

HR & PAYROLL MANAGEMENT PLATFORM

High-Level Design (HLD)

Sprint 2 — Geofence Enforcement, Notifications, Expense Claims & Recruitment/ATS

Baseline

Extends the Sprint 1 MVP (v1.1) modular monolith. This document assumes the domain model, REST conventions, RBAC, and architecture already defined in *HR_Payroll_MVP_Sprint1_HLD_v1.1.pdf* and only restates what Sprint 2 changes or adds.

HLD Item	Sprint 2 Assumption
New domains	Geofence Enforcement, Notifications, Expense Claims, Recruitment/ATS
New role	Hiring Manager (scoped to assigned recruitment pipeline)
New infrastructure	Redis-backed job queue, object storage (receipts, resumes)
Architecture style	Unchanged — modular monolith, versioned REST API, PostgreSQL
Breaking changes	One: <code>Organization.locationCaptureEnabled</code> → <code>locationPolicy</code>

Changelog — v1.0 (Sprint 2)

Initial Sprint 2 design. Adds four additive modules on top of Sprint 1 v1.1. Introduces the platform's first background worker/queue (notifications) and first object storage use (receipts, resumes). See ADR-010 through ADR-015 (§27) for every deliberate deviation.

Table of Contents

1. Document Purpose and Design Goals
2. Business and Product Context
3. Sprint 2 Scope and System Boundary
4. Architecture Overview
5. Logical Architecture
6. Deployment Architecture
7. Domain / Module Architecture
8. Core Data Architecture
9. Entity Relationship Overview
10. Authentication and Authorization Architecture
11. Geofence Enforcement Architecture
12. Notification Architecture
13. Expense Claims Architecture
14. Recruitment / ATS Architecture
15. Candidate-to-Employee Handoff
16. API Architecture
17. Frontend Architecture
18. Transaction and Consistency Strategy
19. Security Architecture
20. Audit and Observability
21. Error Handling and Validation
22. Non-Functional Requirements
23. Scalability and Extension Strategy
24. Backup and Recovery
25. Testing Architecture
26. CI/CD and Environment Strategy
27. Architecture Decision Records
28. Sprint 2 Implementation Roadmap
29. Definition of Done
- Appendix A — New Data Entities
- Appendix B — API Surface (Sprint 2 additions)
- Appendix C — Key State Machines
- Appendix D — Future Architecture

1. Document Purpose and Design Goals

This HLD defines the technical structure for Sprint 2 of the HR & Payroll Management Platform. Sprint 1 delivered the core employee-to-payslip workflow (auth, org/departments, employees, attendance with optional GPS logging, leave, overtime, salary, payroll, payslips, dashboards, audit). Sprint 2 adds four modules that were explicitly deferred at the end of Sprint 1: hard geofence enforcement, a notification layer, expense claims, and an internal recruitment/ATS pipeline.

The modular monolith is preserved. No Sprint 2 module requires extracting a service or changing the deployment topology beyond adding a background worker process and an object storage adapter — both of which Sprint 1's Scalability and Extension Strategy (§21) already anticipated as the natural next additions.

Primary design goals

- Every Sprint 2 feature is additive: no Sprint 1 API contract changes, one narrow schema migration on `Organization`.
- Reuse established patterns instead of inventing new ones: approval workflows mirror Leave/Overtime; payroll-eligible inputs mirror Overtime; sensitive-data handling mirrors Salary/GPS.
- Notifications must never become a transaction dependency — a failed or slow notification can never block or roll back the business action that triggered it.
- Recruitment stays HR-internal in Sprint 2 (no public careers page or candidate login) so the release stays deliverable.
- Geofence enforcement is opt-in per organization and never applies to remote/hybrid employees.

2. Business and Product Context

Sprint 1 proved the HR operations backbone. Sprint 2 responds to four gaps observed after the MVP: (1) some customers require a real access control on office attendance, not just visibility; (2) HR and employees miss approvals because nothing tells them a request changed state; (3) employees have no way to claim work-related expenses; (4) hiring today happens outside the platform, so new joiners are re-typed into Employee records by hand.

Business Area	Sprint 2 Representation
Site access control	Optional geofenced attendance for office-model employees
Operational awareness	In-app and email notifications on approvals, payroll release, expense decisions
Employee reimbursement	Expense claim submission, HR approval, payroll-linked reimbursement
Talent acquisition	Internal job postings, candidate pipeline, interviews, offers, hire conversion

Sprint 2 does not introduce a public-facing website. Job postings and candidate records are entered and managed by HR/Hiring Managers inside the platform (§14, ADR-014).

3. Sprint 2 Scope and System Boundary

In scope

```
Org enables geofence policy + defines zones
↓
Office employee check-in inside/outside a zone → allow / reject with reason
↓
Business event occurs (leave approved, OT approved, payroll released, expense decided)
↓
Notification event recorded → dispatched to in-app inbox + email
↓
Employee submits expense claim + receipt → HR approves/rejects → reimbursed via next payroll run
↓
HR opens a job posting → enters candidates → pipeline stages → interview → offer
↓
Offer accepted → HR converts candidate to Employee (one transaction) → normal Sprint 1 lifecycle takes over
```

- Geofence zones and enforcement policy per organization
- Hard rejection of office check-ins outside configured zones (opt-in)
- Transactional outbox + background dispatcher for notifications
- In-app notification inbox and per-user notification preferences
- Expense categories, claims, receipt upload, HR approval, payroll integration
- Job postings, candidates, pipeline stages, interviews, offers
- Hiring Manager role scoped to assigned pipeline items
- Candidate-to-Employee conversion on offer acceptance

Out of scope for Sprint 2

- Public careers page / candidate self-service portal or login
- Calendar integration for interview scheduling (manual date/time entry only)
- SMS or push notification channels (email + in-app only)
- Standalone reimbursement outside a payroll run (no ad-hoc bank transfer)
- Performance/KPI, learning management, asset management
- Bank disbursement integration, AI HR agent
- Multi-zone geofence shapes beyond circular radius (polygon zones)

Scope control rule (carried from Sprint 1): a feature belongs in Sprint 2 only if it is required to complete one of the four module workflows end-to-end, or to keep the system secure/auditable. Nice-to-have UI polish does not displace domain correctness.

4. Architecture Overview

```
+-----+
| Web Browser / Client                               |
| React + TypeScript + Tailwind                     |
+-----+
      | HTTPS / JSON
      v
+-----+
| Node.js API Application                           |
| Express + TS                                       |
| ...Sprint 1 modules... | Geofence | Expenses | Recruitment |
+-----+
      |
      |
      v
+-----+ +-----+ +-----+
| PostgreSQL | | Object Storage | | Background Worker |
| transactional data | | receipts, resumes | | (notification |
+-----+ +-----+ | dispatcher) |
                        +-----+
                        | Redis (job queue) |<-----+
                        +-----+
```

Two components appear for the first time: a background worker process that drains the notification outbox, and object storage for receipts and resumes. Both were pre-approved extension points in Sprint 1 §21 ("Add Redis when... queues... is justified", "Use object storage for employee documents"). The API remains stateless; the worker is a separate stateless process that can scale independently.

5. Logical Architecture

The same four-layer discipline from Sprint 1 §5 applies unchanged (Controller/Route → Service/Domain → Repository → Infrastructure). The worker process reuses the same domain services as the API — it must never contain its own copy of business logic.

Updated backend module structure

```
src/
  modules/
    ...(Sprint 1 modules: auth, organizations, departments, positions,
      employees, attendance, leave, overtime, salary, payroll,
      payslips, dashboard, audit)
    geofence/           (new — zones + policy enforcement, called from attendance)
    notifications/      (new — outbox, templates, preferences, in-app inbox)
    expenses/           (new — categories, claims, receipts)
    recruitment/        (new — postings, candidates, pipeline, interviews, offers)
  common/
    middleware/
    errors/
    validation/
  infra/
    storage/            (new — object storage adapter, provider-agnostic)
    queue/              (new — Redis/BullMQ adapter)
  workers/
    notification-dispatcher.ts  (new — separate process entry point)
  auth/
  utils/
  database/
  config/
  app.ts
  server.ts
```

Attendance calls into Geofence as a read-only policy check (`geofence.evaluate(employee, coordinates)`); Geofence never calls back into Attendance. This keeps the dependency direction one-way, matching the existing Attendance → Salary/Leave/OT dependency style in Sprint 1 §7.

6. Deployment Architecture

Component	Sprint 2 Change
API instances	Unchanged — stateless, horizontally scalable
Background worker	New. One or more instances consuming the notification queue.
Redis	New. Backs the job queue; not used for caching in Sprint 2.
Object storage	New. S3-compatible bucket for receipts and resumes.
PostgreSQL	Unchanged — remains the single source of truth; outbox table lives here too.

```
Internet
|
v
Reverse Proxy / Load Balancer
|           |
v           v
Web App     API Instances -----> PostgreSQL
           |           |
           |           +-----> Object Storage (receipts, resumes)
           v
        Redis Queue
           |
           v
Notification Worker(s) ---> SMTP / Email Provider
                        ---> PostgreSQL (in-app Notification rows)
```

The worker is optional for local development: if Redis is not configured, notification events still record in the outbox but are not dispatched. This keeps developer setup for the other three modules unaffected.

7. Domain / Module Architecture

Module	Owns	Depends On
Geofence	Zones, organization location policy	Organization
Attendance	(unchanged) work sessions + GPS metadata	Employee, Geofence (policy check)
Notifications	Outbox events, templates, preferences, in-app inbox	User, all event-emitting modules
Expenses	Categories, claims, receipts	Employee, Object Storage, Payroll (input)
Recruitment	Job postings, candidates, pipeline, interviews, offers	Department, Position, Object Storage
Employee	(unchanged, extended) also accepts creation from an accepted Offer	Recruitment (conversion trigger only)

Notifications depends on every event-emitting module, but no module may depend back on Notifications for its own correctness. A leave approval must succeed and commit even if the notification outbox insert somehow failed — see §18.

8. Core Data Architecture

The Sprint 1 principles (§8) are unchanged and apply to every new table: UUID primary keys, DECIMAL/NUMERIC for money, DATE for calendar fields, status/deactivation over hard deletes, organization_id indexed everywhere. Two additions:

Principle	Design
Outbox durability	<code>notification_events</code> is a normal table in the same PostgreSQL database and the same transaction as the triggering write — never a separate message store.
File references	Receipts and resumes store an object storage key + metadata (filename, mime type, size) in PostgreSQL; the binary never lives in the relational database.

9. Entity Relationship Overview

```
Organization
|--- GeofenceZone (label, lat, lng, radiusMeters)
|--- (existing Sprint 1 tree unchanged)

Employee
|--- ExpenseClaim --- ExpenseCategory
|       |--- ExpenseReceipt (object storage key)
|--- (existing attendance/leave/OT/salary unchanged)

User
|--- Notification (recipient)
|--- NotificationPreference

Organization
|--- NotificationEvent (outbox)
|--- NotificationTemplate

Organization
|--- JobPosting --- Department / Position
|       |--- Candidate
|           |--- CandidateApplication (pipeline stage)
|               |--- Interview
|               |--- Offer --- (on ACCEPTED) ---> Employee

All sensitive operations ---> AuditLog
```

An accepted `Offer` produces exactly one `Employee` + `User` pair via the same creation path HR already uses for manual onboarding — `Employee` does not gain a second, parallel creation code path (§15, ADR-015).

10. Authentication and Authorization Architecture

Sprint 2 adds one role: **Hiring Manager**. It is scoped, not global — a Hiring Manager only sees the postings/candidates/interviews they are explicitly assigned to.

Role	Geofence	Notifications	Expenses	Recruitment
Super Admin	Full	Full	Full	Full
HR Admin	Manage zones/policy	Manage templates; own inbox	Approve/reject; manage categories	Full
Hiring Manager	—	Own inbox	Own claims only	Assigned postings/candidates: view + interview feedback only. No offer creation, no salary visibility.
Employee	—	Own inbox	Own claims: submit/view	—

Authorization flow (HTTP → authenticate → resolve org → check role/permission → validate ownership/scope → execute → audit if sensitive → respond) is unchanged from Sprint 1 §10. Recruitment scope adds one extra check: resource-level assignment ownership for Hiring Manager, on top of the existing role check.

11. Geofence Enforcement Architecture

This section implements the deferred design from Sprint 1 HLD v1.1 Appendix D and closes ADR-009's stated future extension.

11.1 Policy model

`Organization.locationPolicy` replaces the Sprint 1 v1.1 boolean `locationCaptureEnabled`:

Value	Behavior
NONE	No coordinates captured or stored (old <code>locationCaptureEnabled=false</code>)
LOG_ONLY	Coordinates captured and stored, never gate check-in (old <code>locationCaptureEnabled=true</code> , Sprint 1 v1.1 default)
GEOFENCE_ENFORCED	Coordinates captured; check-in is rejected if the employee is <code>work_model=OFFICE</code> and outside every configured zone

11.2 Enforcement sequence

```
Employee → POST /attendance/check-in { lat?, lng?, accuracyMeters? }
↓
Load organization.locationPolicy and employee.work_model
↓
policy != GEOFENCE_ENFORCED OR work_model != OFFICE?
  → proceed exactly as Sprint 1 §11 (location is informational only)
  ↓ (otherwise)
Coordinates missing? → reject 422 LOCATION_REQUIRED
↓
Coordinates outside every active GeofenceZone for the org? → reject 422
OUTSIDE_GEOFENCE
↓
Otherwise create AttendanceRecord exactly as Sprint 1 §11
```

11.3 Rules

Rule	Implementation
Scope	Enforcement applies only to <code>work_model=OFFICE</code> ; hybrid/remote employees are always exempt, regardless of policy.
Mandatory location	Under GEOFENCE_ENFORCED, coordinates become mandatory for in-scope employees — the Sprint 1 "always optional" rule (ATT-09) is narrowed only for this case.
Multiple zones	An org may define multiple circular zones (e.g. two offices); a check-in inside any one zone passes.
Zone shape	Circular only (center lat/lng + radius meters) in Sprint 2; polygon zones deferred (Appendix D).

Rejected attempts	Not persisted as an AttendanceRecord; logged to AuditLog as <code>attendance.checkin.rejected_geofence</code> for HR troubleshooting.
Check-out	Not geofenced in Sprint 2 — only check-in is gated, matching that check-out already requires an existing open session.

Design rationale: narrowing enforcement to OFFICE employees only, and to check-in only, keeps the blast radius of a wrong reading small — a stale GPS fix can at worst force a retry for an office worker already on-site, never block a legitimate remote check-in. This preserves the Sprint 1 goal of "check in from everywhere" for everyone it was meant for.

12. Notification Architecture

12.1 Outbox pattern

Every domain action that should notify someone writes a row to `notification_events` inside its own existing database transaction — no new transaction is opened for this. A separate background worker polls for `PENDING` events, renders them against a `NotificationTemplate`, and:

- Inserts a `Notification` row (in-app) for each recipient
- Enqueues an email send via an SMTP/provider adapter
- Marks the event `PROCESSED`, or `FAILED` after exhausting retries with backoff

```
Business action commits (e.g. leave approved)
  ↓ (same transaction)
Insert NotificationEvent{ eventType, orgId, payload, status=PENDING }
  ↓ (transaction commits – business action is now durable regardless of notification
outcome)
Worker polls PENDING events
  ↓
Render template → create Notification row(s) → attempt email send
  ↓
Success → PROCESSED | Failure → retry with backoff, then FAILED after max attempts
```

12.2 Event catalog (Sprint 2)

Event	Recipients
leave.request.submitted	HR Admin(s)
leave.request.decided	Requesting employee
overtime.request.decided	Requesting employee
payroll.run.released	All employees in the run (payslip available)
expense.claim.decided	Submitting employee
recruitment.interview.scheduled	Assigned Hiring Manager(s)
recruitment.offer.responded	HR Admin who created the offer

12.3 Rules

Rule	Requirement
Non-blocking	A notification failure never rolls back or delays the triggering business transaction (§18).
Idempotent delivery	Worker processing is at-least-once; each event carries a unique id so re-processing a event does not duplicate an in-app Notification row.

No sensitive payloads in email	Emails link back into the app; they never include salary amounts, expense amounts, or GPS coordinates in the subject or body.
Preferences	Each user can disable email per event type; in-app notifications cannot be fully disabled (HR must always be reachable for approvals).
Degrade gracefully	If Redis/worker is unavailable, events accumulate in the outbox and are drained once the worker returns — no data loss.

13. Expense Claims Architecture

13.1 Flow

```
Employee → POST /expenses/claims (categoryId, amount, expenseDate, description)
↓
Employee → POST /expenses/claims/:id/receipts (multipart upload → object storage)
↓
Employee → POST /expenses/claims/:id/submit (DRAFT → SUBMITTED)
↓
HR → POST /expenses/claims/:id/approve | /reject
    ↓ (APPROVED only)
Included as a payroll input on the next PayrollRun for that employee/period
↓
PayrollItem snapshot records the reimbursed amount → claim status REIMBURSED
```

13.2 Rules

Rule	Requirement
Ownership	Only the submitting employee may edit a DRAFT claim; nobody may edit after SUBMITTED except HR via reject-with-reason.
Receipt requirement	Categories may be flagged <code>requiresReceipt</code> ; submission is rejected (422 <code>RECEIPT_REQUIRED</code>) if none is attached.
Approval authority	Only HR Admin/Super Admin may approve or reject — same authority tier as leave/OT approval.
Payroll eligibility	Only APPROVED claims for the payroll period are payroll-eligible — mirrors OT-05 from Sprint 1.
Immutability after inclusion	A claim included in a calculated/approved PayrollRun cannot be edited or re-decided — mirrors OT-06.
Currency	Claims are recorded in the organization's payroll currency in Sprint 2 (no multi-currency conversion).

Expense reuses the exact "approved-only, snapshotted-at-calculation" pattern Sprint 1 established for Overtime rather than creating an independent reimbursement ledger (ADR-013). This is a deliberate reuse, not a new concept.

13.3 Receipt storage

Concern	Design
Storage	Object storage (S3-compatible); PostgreSQL stores the key + filename + mime type + size only.

Allowed types	image/jpeg, image/png, application/pdf
Size limit	10 MB per file, up to 5 files per claim
Access	Signed, time-limited URLs issued per request; never a permanently public bucket path.
Validation	Shape-only (extension/mime/size); no content/malware scanning in Sprint 2 — documented as an accepted limitation.

14. Recruitment / ATS Architecture

14.1 Flow

```
HR → POST /recruitment/postings (title, department, position, openings)
↓
HR/Recruiter → POST /recruitment/postings/:id/candidates (manual entry + resume upload)
↓
CandidateApplication created at stage = APPLIED
↓
HR/Hiring Manager moves stage: APPLIED → SCREENING → INTERVIEW → OFFER
↓
Interview(s) scheduled + feedback recorded by assigned Hiring Manager
↓
HR → POST /recruitment/applications/:id/offer (salary, start date)
↓
Candidate response recorded by HR: ACCEPTED | DECLINED
↓ (ACCEPTED)
HR → POST /recruitment/applications/:id/convert → creates Employee + User ($15)
```

14.2 Rules

Rule	Requirement
No public entry point	Candidates are entered by HR/Recruiter only; there is no public application form or candidate login in Sprint 2.
Stage ordering	Stage transitions are forward-only except explicit REJECTED/WITHDRAWN, which are terminal from any stage.
Offer authority	Only HR Admin/Super Admin may create or rescind an offer — Hiring Manager cannot see or set compensation.
Single active offer	An application may have at most one non-terminal (DRAFT/SENT) offer at a time.
Duplicate applications	The same candidate email cannot have two open (non-terminal) applications to the same posting — reject with 409.
Resume storage	Same object storage adapter and access rules as expense receipts (§13.3); resumes readable by HR Admin and the assigned Hiring Manager only.

14.3 Hiring Manager scope

A Hiring Manager is assigned to specific postings/applications by HR. Their access is limited to: viewing assigned candidate profiles (excluding compensation history), scheduling/recording interview feedback, and advancing/rejecting a stage up to but not including OFFER. This is the first Sprint 2 role that requires resource-level (not just type-level) authorization — see §10.

15. Candidate-to-Employee Handoff

Converting an accepted offer into an active Employee is the single integration point between Recruitment and the rest of the platform, and the highest-risk transaction in Sprint 2.

```
HR → POST /recruitment/applications/:id/convert
  ↓ (single database transaction)
Validate: offer.status == ACCEPTED, application not already converted
  ↓
Create User (role=EMPLOYEE, status=ACTIVE, prefilled email from candidate)
  ↓
Create Employee (prefilled name/department/position; salary profile from offer terms)
  ↓
Set CandidateApplication.stage = HIRED, link convertedEmployeeId
  ↓
Write AuditLog entry (actor, candidateId, employeeId)
  ↓
Commit — enqueue notification event (new employee account ready) after commit
```

This is the one place Sprint 2 touches the Employee domain's write path. Recruitment must call the same Employee-creation service Sprint 1's HR onboarding UI uses — never duplicate employee-creation logic inside the Recruitment module (ADR-015). If any step fails, the entire conversion rolls back: no orphan User without an Employee, and no Employee without a linked User.

16. API Architecture

Sprint 2 follows the exact REST conventions from Sprint 1 §14: versioned paths under `/api/v1`, the same success/error envelope, and the same HTTP status vocabulary (§21). See Appendix B for the full endpoint list.

Domain	Representative Endpoint
Geofence	GET/PATCH <code>/api/v1/organization/geofence-policy</code> ; GET/POST <code>/api/v1/organization/geofence-zones</code>
Notifications	GET <code>/api/v1/notifications</code> ; PATCH <code>/api/v1/notifications/:id/read</code> ; GET/PATCH <code>/api/v1/notification-preferences</code>
Expenses	POST <code>/api/v1/expenses/claims</code> ; POST <code>/api/v1/expenses/claims/:id/receipts</code> ; POST <code>/api/v1/expenses/claims/:id/approve</code>
Recruitment	POST <code>/api/v1/recruitment/postings</code> ; POST <code>/api/v1/recruitment/postings/:id/candidates</code> ; POST <code>/api/v1/recruitment/applications/:id/offer</code>
Handoff	POST <code>/api/v1/recruitment/applications/:id/convert</code>

Response envelope and error shape are unchanged from Sprint 1 §14 — new error `code` values are added (§21) but the envelope itself is not versioned separately.

17. Frontend Architecture

```
src/  
  features/  
    ...(Sprint 1 features unchanged)  
    geofence/           (org settings: policy + zone management)  
    notifications/      (bell/inbox, preferences)  
    expenses/           (employee submit, HR approve)  
    recruitment/        (postings, pipeline board, candidate detail, offers)
```

Concern	Recommendation
Notification bell	Poll or lightweight subscribe for unread count; do not introduce a WebSocket layer for Sprint 2 — polling is sufficient at MVP scale.
File upload	Direct-to-API multipart upload; API forwards to object storage. Do not expose storage credentials to the browser.
Pipeline board	Simple column-per-stage list view is sufficient; drag-and-drop is not required for Sprint 2.
Geofence map	A simple lat/lng/radius form is sufficient for zone entry; an interactive map picker is a nice-to-have, not required.

18. Transaction and Consistency Strategy

Operation	Transaction Boundary
Approve expense claim	Status update + audit + notification-event insert, one transaction
Reject geofenced check-in	No AttendanceRecord write; audit log entry only, its own small transaction
Convert candidate to employee	User + Employee + application status + audit, one transaction (§15)
Calculate payroll (extended)	Now also loads APPROVED expense claims for the period, same snapshot boundary as Sprint 1 §13
Notification dispatch	Deliberately outside the triggering transaction — see §12.1

Never make the triggering business transaction depend on notification delivery succeeding. If the outbox insert itself is in the same transaction and that insert fails, the whole business transaction correctly rolls back — but a slow SMTP server or a worker outage must never be able to fail a leave approval.

19. Security Architecture

Risk	Control
Geofence bypass via omitted coordinates	Coordinates become mandatory (not optional) for in-scope employees under GEOFENCE_ENFORCED (§11.3).
Zone data tampering	Only HR Admin/Super Admin may create/edit zones; changes are audited.
Notification content leakage	Salary, expense amounts, and GPS data never appear in email bodies/subjects — link-back only (§12.3).
Receipt/resume exposure	Signed, time-limited URLs only; access checked against the same ownership/role rules as the parent record.
Candidate PII exposure	Hiring Manager sees only assigned candidates and never compensation history or offer terms.
Upload abuse	Size and mime-type limits enforced server-side, not just in the UI (§13.3).
Orphaned accounts from a failed conversion	Single-transaction conversion (§15) — a partial failure leaves no User/Employee at all.

20. Audit and Observability

Audit at minimum, in addition to Sprint 1's list (§18): geofence zone create/update/delete; organization `locationPolicy` changes; rejected geofenced check-in attempts; expense claim approval/rejection; job posting create/close; offer creation/rescission/response; candidate-to-employee conversion.

Operational observability adds: notification outbox depth (pending count) and oldest-pending age as a health signal for the worker; worker job success/failure/retry counts.

21. Error Handling and Validation

Code	Meaning
LOCATION_REQUIRED (422)	Coordinates missing on check-in while GEOFENCE_ENFORCED applies to this employee
OUTSIDE_GEOFENCE (422)	Coordinates present but outside every configured zone
RECEIPT_REQUIRED (422)	Expense category requires a receipt and none was attached
EXPENSE_ALREADY_DECIDED (409)	Attempt to approve/reject/edit a claim not in SUBMITTED state
DUPLICATE_APPLICATION (409)	Candidate already has an open application to the same posting
OFFER_ALREADY_RESPONDED (409)	Attempt to record a response on a non-SENT offer
CONVERSION_NOT_ELIGIBLE (409)	Convert attempted without an ACCEPTED offer, or already converted

These extend, and use the same HTTP status vocabulary as, Sprint 1 §19.

22. Non-Functional Requirements

Category	Sprint 2 Target
Notification latency	In-app: available within ~60s of the triggering action under normal load. Email: best-effort, not guaranteed delivery time.
Upload limits	10 MB/file for receipts, 5 MB for resumes; enforced server-side.
Worker resilience	Worker restart must not lose or duplicate pending outbox events (idempotent, at-least-once).
Geofence check cost	Zone lookup adds negligible latency to check-in (<10ms typical, small org-scoped zone count).
Backward compatibility	Sprint 1 v1.1 clients that never send a location policy continue to work under the migrated default (§27, ADR-010).

23. Scalability and Extension Strategy

- Redis, introduced here for the notification queue, is the same instance later work (rate limiting, caching) can reuse — do not stand up a second queue technology.
- Object storage, introduced here for receipts/resumes, is the same adapter later modules (assets, documents) should reuse.
- Geofence zones are per-organization and small in count; no need for spatial indexing at Sprint 2 scale (radius-distance check is sufficient).
- The notification event catalog (§12.2) is designed to grow — adding a new event type must never require a schema migration, only a new template row.

Future	Integration Point
Polygon geofence zones	Geofence module, additive zone-shape field
SMS/push notifications	Notification dispatcher, new channel adapter
Public careers page / candidate portal	Recruitment module, new unauthenticated intake endpoint
Calendar integration	Interview scheduling, external adapter
Standalone reimbursement	Expenses module, decoupled from payroll cycle

24. Backup and Recovery

- PostgreSQL remains the single authoritative store for all Sprint 2 relational data, including the notification outbox — unchanged posture from Sprint 1 §22.
- Object storage (receipts, resumes) needs its own backup/versioning policy — define retention before production use; do not assume the storage provider's default durability equals a backup.
- Redis holds only transient queue state (already-durable outbox rows are the source of truth); losing Redis loses in-flight jobs, not data — the worker must re-scan PENDING events on restart.

25. Testing Architecture

Test Layer	Examples
Unit	Geofence distance calculation, expense reimbursement snapshot, offer state transitions
Service	Approve expense, reject geofenced check-in, convert candidate to employee
Repository	Organization scoping on new tables, uniqueness constraints (duplicate application)
API integration	RBAC for Hiring Manager scope, receipt upload validation, notification preference toggles
E2E	Posting → candidate → interview → offer → accept → convert → new employee can log in
Worker	Outbox drains exactly once per event; failure retried with backoff; worker restart does not duplicate deliveries

Critical invariants

- A GEOFENCE_ENFORCED office employee cannot check in from outside every configured zone.
- A hybrid/remote employee is never subject to geofence rejection, regardless of organization policy.
- An expense claim can be reimbursed at most once, and only after HR approval.
- An application can be converted to an employee at most once.
- A Hiring Manager cannot read another Hiring Manager's assigned candidates or any compensation data.
- A failed notification dispatch never rolls back the business transaction that generated it.

26. CI/CD and Environment Strategy

```
Pull Request
↓
Lint → Typecheck → Unit Tests
↓
Build → Integration Tests (incl. worker + storage adapter, mocked)
↓
Container Build (API image + Worker image)
↓
Development (Postgres + Redis + local storage emulator)
↓
Staging / Release Candidate (real object storage bucket, real SMTP sandbox)
↓
Production Approval
↓
Deploy API + Worker + Migration + Health Check
```

The worker ships as a second container image built from the same repository/commit as the API, so the two are always deployed in lockstep — never let the worker run business logic from a different code version than the API.

27. Architecture Decision Records

ADR	Decision	Reason
ADR-010	Geofence enforcement is opt-in per organization and scoped to OFFICE work-model employees only	Extends ADR-009; avoids blocking legitimate remote/hybrid check-ins
ADR-011	Notifications use a transactional outbox + background worker, never a synchronous send	Decouples business transaction success from email/SMTP availability
ADR-012	Introduce Redis-backed job queue for notification dispatch	First justified use of Redis per Sprint 1 §21; reusable by future background work
ADR-013	Expense claims reuse the approved-only, payroll-snapshotted pattern from Overtime	Avoids a second, divergent reimbursement mechanism
ADR-014	Recruitment is HR/recruiter-entered only in Sprint 2; no public apply flow	Keeps scope deliverable; public intake deferred to Appendix D
ADR-015	Candidate-to-Employee conversion calls the existing Employee creation service	Prevents a duplicated, divergent employee-creation code path

28. Sprint 2 Implementation Roadmap

Order	Work Package	Deliverable
1	Infra foundation	Redis, object storage adapter, migrations for new tables
2	Geofence	Zones + policy schema, enforcement in check-in, org settings UI
3	Notifications core	Outbox schema, templates, worker, in-app inbox API
4	Notification wiring	Emit events from leave/OT/payroll; frontend bell + preferences
5	Expenses	Categories, claims, receipt upload, HR approval, frontend
6	Expenses–payroll link	Approved claims as payroll input; snapshot in PayrollItem
7	Recruitment core	Postings, candidates, pipeline, interviews schema + API + frontend
8	Offers & handoff	Offer lifecycle, candidate-to-employee conversion, notification wiring
9	Hardening	Audit coverage, RBAC tests for Hiring Manager scope, security review
10	Release	Staging with real storage/queue, migration, demo script

Build vertically per module (schema → service → API → frontend) rather than all schemas first — this exposes cross-module integration issues (notification wiring, payroll linkage) early, as in Sprint 1 §25.

29. Definition of Done

- All Sprint 2 modules have authenticated and authorized APIs.
- Migrations apply cleanly on top of Sprint 1 v1.1 schema, including the `locationPolicy` migration.
- Geofence enforcement rejects out-of-zone office check-ins and never affects hybrid/remote employees.
- A failed or delayed notification never blocks or rolls back its triggering business action.
- Approved expense claims appear correctly in the next payroll run's calculation and cannot be double-reimbursed.
- A full pipeline (posting → candidate → interview → offer → accept → convert) produces a working Employee/User login.
- Hiring Manager cannot access unassigned candidates, compensation data, or offer terms.
- Receipts and resumes are only reachable via signed, access-checked URLs.
- Sensitive Sprint 2 actions are audited (§20).
- CI passes lint, tests, and build for both the API and worker images.
- Staging deployment succeeds with real Redis and object storage configuration.
- End-to-end demo covers all four modules plus the candidate-to-employee handoff.

Appendix A — New Data Entities

Entity	Key Fields / Notes
Organization (extended)	location_policy (NONE LOG_ONLY GEOFENCE_ENFORCED) replaces location_capture_enabled
GeofenceZone	id, organization_id, label, lat, lng, radius_meters, status
NotificationTemplate	id, organization_id?, event_type, channel, subject, body_template
NotificationEvent	id, organization_id, event_type, payload (json), status (PENDING/PROCESSED/FAILED), attempts, created_at
Notification	id, organization_id, user_id, title, message, related_resource_type, related_resource_id, read_at, created_at
NotificationPreference	id, user_id, event_type, email_enabled
ExpenseCategory	id, organization_id, name, requires_receipt, status
ExpenseClaim	id, organization_id, employee_id, category_id, amount, expense_date, description, status, payroll_item_id, approved_by, approved_at
ExpenseReceipt	id, expense_claim_id, storage_key, file_name, mime_type, size_bytes
JobPosting	id, organization_id, title, department_id, position_id, employment_type, openings, status
Candidate	id, organization_id, full_name, email, phone, resume_storage_key, source, converted_employee_id
CandidateApplication	id, candidate_id, job_posting_id, stage, hiring_manager_id, stage_updated_at
Interview	id, application_id, scheduled_at, mode, interviewer_ids, feedback, score, status
Offer	id, application_id, proposed_salary, currency, start_date, status, sent_at, responded_at

Appendix B — API Surface (Sprint 2 additions)

Domain	Method	Path
Geofence	GET/PATCH	/api/v1/organization/geofence-policy
Geofence	GET/POST	/api/v1/organization/geofence-zones
Geofence	PATCH/DELETE	/api/v1/organization/geofence-zones/:id
Notifications	GET	/api/v1/notifications
Notifications	PATCH	/api/v1/notifications/:id/read
Notifications	GET/PATCH	/api/v1/notification-preferences
Expenses	GET/POST	/api/v1/expenses/categories
Expenses	GET/POST	/api/v1/expenses/claims
Expenses	POST	/api/v1/expenses/claims/:id/receipts
Expenses	POST	/api/v1/expenses/claims/:id/submit
Expenses	POST	/api/v1/expenses/claims/:id/approve /reject
Recruitment	GET/POST	/api/v1/recruitment/postings
Recruitment	POST	/api/v1/recruitment/postings/:id/candidates
Recruitment	PATCH	/api/v1/recruitment/applications/:id/stage
Recruitment	POST	/api/v1/recruitment/applications/:id/interviews
Recruitment	POST	/api/v1/recruitment/applications/:id/offer
Recruitment	POST	/api/v1/recruitment/applications/:id/offer/respond
Recruitment	POST	/api/v1/recruitment/applications/:id/convert

Appendix C — Key State Machines

Expense Claim

```
DRAFT → SUBMITTED → APPROVED → REIMBURSED
      |
      --> REJECTED
DRAFT/SUBMITTED --> CANCELLED (by employee, before decision)
```

Candidate Application (pipeline stage)

```
APPLIED → SCREENING → INTERVIEW → OFFER → HIRED
      |           |           |           |
      +-----+-----+-----+---> REJECTED / WITHDRAWN (terminal, any stage)
```

Offer

```
DRAFT → SENT → ACCEPTED → (convert) → HIRED
      |
      +--> DECLINED
DRAFT/SENT --> RESCINDED
```

Notification Event

```
PENDING → PROCESSED
      |
      --> FAILED (after max retries with backoff)
```

Appendix D — Future Architecture

Deferred beyond Sprint 2:

- Public careers page / candidate self-service portal with candidate login
- Polygon-shaped geofence zones (beyond circular radius)
- SMS/push notification channels
- Calendar integration for interview scheduling
- Standalone reimbursement outside the payroll cycle
- Performance/KPI, learning management, asset management
- Bank disbursement integration, AI HR agent (as in Sprint 1 Appendix D)

End of HLD — Sprint 2 v1.0